Tema 4: Interfaces Gráficas con Swing
Más Componentes

Pablo Gordillo (pabgordi@ucm.es)

Departamento de Sistemas Informáticos y Computación

Facultad de Informática



(Basado en el material creado por Puri Arenas, Jesús Correas y Samir Genaim)

JCheckBox

- Es un componente que se puede seleccionar/deseleccionar y que muestra su estado al usuario.
- Se pueden tener tantos **checkBox** como se deseen y **puede haber varios seleccionados simultáneamente**.
- Al seleccionar con el ratón un **checkBox** se generan un **ItemEvent** y un **ActionEvent**. Los eventos de tipo **Item** se pueden modificar a través del código. Siempre que se modifican vía usuario o código se ejecuta el método **ItemStateChanged(ItemEvent e)**. EL listener se añade con:

  checkBox.addItemListener(this);
- Para marcar un **checkBox** desde el código, se usa el método

  checkBox.setSelected(boolean);
- Se puede saber si el item está seleccionado

  itemEvent.getStateChange() == ItemEvent.SELECTED/ItemEvent.DESELECTED
- Para saber que item ha sufrido un cambio (seleccionado/deseleccionado):

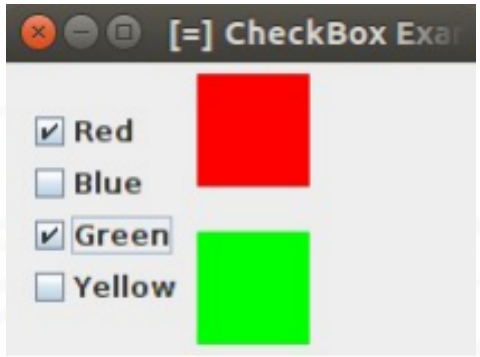
  itemEvent.getItemSelectable() == checkbox

JCheckBox

```
public class CheckBox extends JFrame implements ItemListener {
  JCheckBox redCB, blueCB, greenCB, yellowCB;
  JPanel redBox, blueBox, greenBox, yellowBox;

  private void initGUI() {
    ...
    redCB = new JCheckBox("Red");
    redCB.setSelected(true);
    redCB.addItemListener(this);
    ...
    blueCB = new JcheckBox("Blue");
    blueCB.setSelected(false);
    blueCB.addItemListener(this);
    ...
  }

  public void itemStateChanged(ItemEvent e) {
    if (e.getStateChange() == ItemEvent.DESELECTED) visible = false;
    else visible = true;
    if (e.getItemSelectable() == redCB) redBox.setVisible(visible);
    else if (e.getItemSelectable() == blueCB) blueBox.setVisible(visible);
    else if (e.getItemSelectable() == greenCB) greenBox.setVisible(visible);
    else if (e.getItemSelectable() == yellowCB)
    yellowBox.setVisible(visible);
  }
```

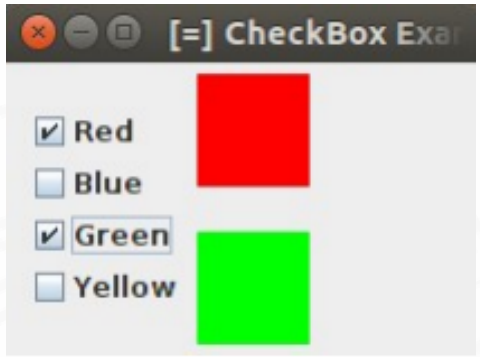


JCheckBox

```
public class CheckBox extends JFrame implements ItemListener {
  JCheckBox redCB, blueCB, greenCB, yellowCB;
  JPanel redBox, blueBox, greenBox, yellowBox;

  private void initGUI() {
    ...
    redCB = new JCheckBox("Red");
    redCB.setSelected(true);
    redCB.addItemListener(this);
    ...
    blueCB = new JcheckBox("Blue");
    blueCB.setSelected(false);
    blueCB.addItemListener(this);
    ...
  }

  public void itemStateChanged(ItemEvent e) {
    if (e.getStateChange() == ItemEvent.DESELECTED) visible = false;
    else visible = true;
    if (e.getItemSelectable() == redCB) redBox.setVisible(visible);
    else if (e.getItemSelectable() == blueCB) blueBox.setVisible(visible);
    else if (e.getItemSelectable() == greenCB) greenBox.setVisible(visible);
    else if (e.getItemSelectable() == yellowCB)
    yellowBox.setVisible(visible);
  }
```



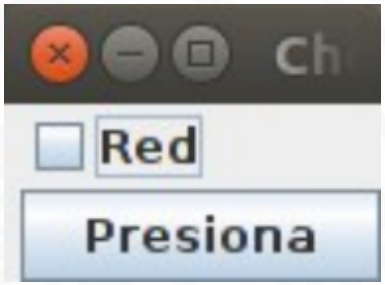
Para agrupar los botones se está usando un BoxLayout

ActionListener vs ItemListener

- La diferencia entre **action** e **item** radica en que los **action** solo reaccionan antes acciones del usuario (no del programador), mientras que los **item** reaccionan tanto a acciones del usuario como del programador.

```
...
private   JCheckBox red;
private   JButton press;
...

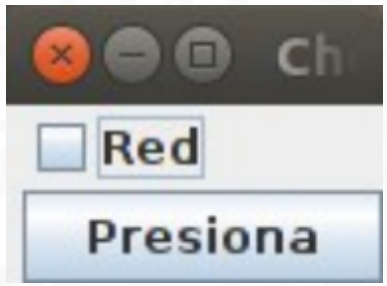
this.red = new JCheckBox("Red");
this.red.addItemListener(this);
this.red.addActionListener(this);
this.press = new JButton("Presiona");
this.press.addActionListener(this);
```



ActionListener vs ItemListener

```
@Override
public void actionPerformed(ActionEvent e) {
  if (e.getSource()==this.press) {
    if (this.red.isSelected())
    this.red.setSelected(false);
    else this.red.setSelected(true);
  }
  else if (e.getSource()==this.red)
  System.out.println("Accion sobre checkbox");
}

@Override
public void itemStateChanged(ItemEvent e) {
  if (e.getItemSelectable() == this.red){
    if (e.getStateChange()==ItemEvent.SELECTED)
    System.out.println("CheckBox seleccionado");
    else if (e.getStateChange()==ItemEvent.DESELECTED)
    System.out.println("CheckBox no seleccionado");
  }
}
```



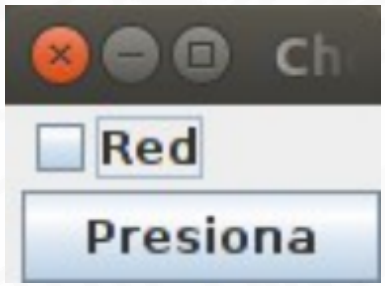
ActionListener vs ItemListener

- Si presionamos el botón **Presiona** se ejecuta el método **actionPerformed**, y cambia el estado del **CheckBox** desde el código. Al cambiar el estado del **CheckBox** desde el código, se ejecuta el método **itemStateChanged** y aparece el **println** correspondiente.
  En este caso el usuario solo ha tocado el botón, no el checkbox, pero se ha ejecutado **itemStateChanged**, obteniendo como salida:

  checkBox seleciconado

- Si el usuario marca o desmarca el **CheckBox**, se ejecuta primero el método **itemStateChanged** y después el método **actionPerformed** dando como salida:

  checkBox seleccionado Accion sobre checkbox



JRadioButton

- Son componentes similares a los **checkBox**, salvo que cambia el aspecto.
- Para evitar que se seleccionen varias, estas componentes se agrupan usando **ButtonGroup**, de forma que solo pueda haber una marcada.
- Un click sobre un **RadioButton** lanza eventos del tipo **Action** e **Item**. Por lo tanto se pueden tratar dentro del método **actionPerformed(actionEvent e)** o **itemStateChanged(ItemEvent e)**.
- La apariencia es distinta: los **checkBox** son cuadrados y los **radioButton** son redondos.

JRadioButton

```
JRadioButton redCB, blueCB, greenCB, yellowCB;
JPanel redBox, blueBox, greenBox, yellowBox;

private void initGUI() {
  ...
  redCB = new JRadioButton("Red");
  redCB.addItemListener(this);
  blueCB = new JRadioButton("Blue");
  blueCB.addItemListener(this);
  ...

  ButtonGroup btnGroup = new ButtonGroup();
  btnGroup.add(greenCB);
  btnGroup.add(yellowCB);
  btnGroup.add(redCB);
  btnGroup.add(blueCB);
  ...
}
```





El ButtonGroup no se añade al panel, solo los RadioButton

JRadioButton

```
JRadioButton redCB, blueCB, greenCB, yellowCB;
JPanel redBox, blueBox, greenBox, yellowBox;

private void initGUI() {
  ...
  redCB = new JRadioButton("Red");
  redCB.addItemListener(this);
  blueCB = new JRadioButton("Blue");
  blueCB.addItemListener(this);
  ...

  ButtonGroup btnGroup = new ButtonGroup();
  btnGroup.add(greenCB);
  btnGroup.add(yellowCB);
  btnGroup.add(redCB);
  btnGroup.add(blueCB);
  ...
}
```

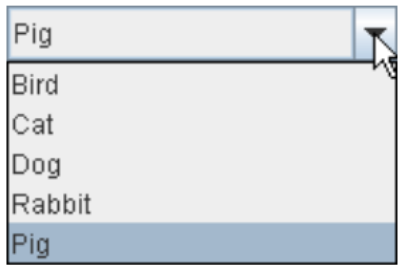




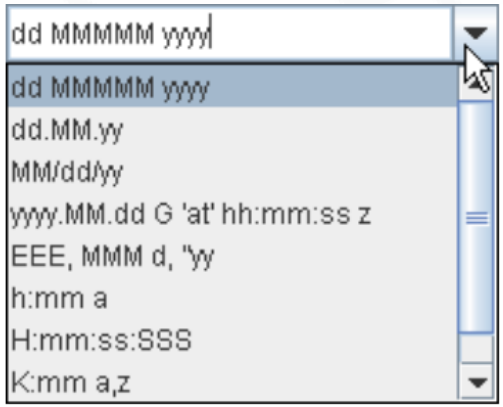

```
  public void itemStateChanged(ItemEvent e) {
    if (e.getStateChange() == ItemEvent.DESELECTED) visible = false;
    else visible = true;
    if (e.getItemSelectable() == redCB) redBox.setVisible(visible);
    else if (e.getItemSelectable() == blueCB) blueBox.setVisible(visible);
    else if (e.getItemSelectable() == greenCB) greenBox.setVisible(visible);
    else if (e.getItemSelectable() == yellowCB) yellowBox.setVisible(visible);
  }
...
```

JComboBox

- Permite al usuario elegir opciones de una lista desplegable.
- Un **ComboBox** puede ser editable o no editable. Por defecto es no editable.
- La forma editable ofrece un campo de texto donde el usuario puede escribir la opción deseada.



No editable



editable

JComboBox

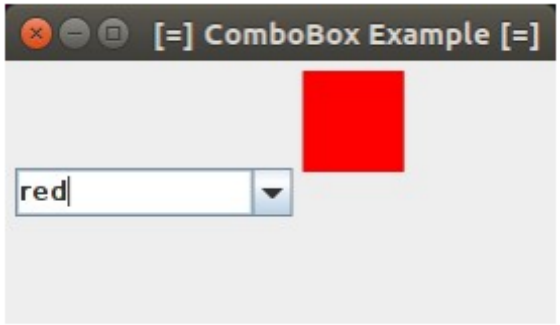
- En la versión editable, el campo de texto sirve para encontrar la opción de la lista de forma más rápida.
- Aunque su funcionalidad es similar a los **RadioButton**, ocupan menos espacio en la ventana. Por tanto son útiles cuando el número de opciones es elevado.
- Cuando el usuario selecciona una opción, se genera un **ActionEvent**.
- Un **ComboBox** se crea pasando como argumento la lista de opciones que queremos que aparezca en el despegable.

JComboBox

```
JComboBox<String> list;
String colors [] = {"blue","red","yellow","green"};
JPanel redBox, blueBox, greenBox, yellowBox;

private void initGUI() {
  ...
  list = new JComboBox<String>(colors);
  list.setSelectedIndex(0);
  list.setEditable(true);
  list.addActionListener(this);
  ...
}

@Override
public void actionPerformed(ActionEvent e) {
  redBox.setVisible(false);
  ...
  if (list.getSelectedItem().equals("red")) redBox.setVisible(true);
  else if (list.getSelectedItem().equals("green"))
         greenBox.setVisible(true);
  else if (list.getSelectedItem().equals("yellow"))
         yellowBox.setVisible(true);
  else if (list.getSelectedItem().equals("blue")) {
         blueBox.setVisible(true);
  }
}
```



JComboBox

- Se pueden añadir y eliminar opciones de un **ComboBox** de forma dinámica con:

```
list.addItem(...);
list.removeItem(...);
```

- Para hacer el **ComboBox** editable basta añadir:

```
list.setEditable(true);
```

JTextArea

- Es un área donde se puede escribir texto organizado en líneas.
- Se puede determinar el tamaño del área fijando el número de filas y columnas. Concretamente se puede fijar en la constructora utilizando:

  new JTextArea(filas, columnas)

- Por defecto no tienen asociado un **scroll**. Más tarde veremos como se puede añadir.
- Se puede fijar si el área de texto es editable (el usuario puede escribir) o no.

  textarea.setEditable(boolean)

- Se puede poner texto usando:

  textarea.setText(String)

JTextArea

- Es posible fijar la fuente deseada con:

  textarea.setFont(new Font("Serif", Font.ITALIC,16));

- Es posible determinar si se quiere que las líneas se partan o que se escriban en una única línea.

  textarea.setLineWrap(boolean);

- Además si se desea que no se corten las palabras, basta poner:

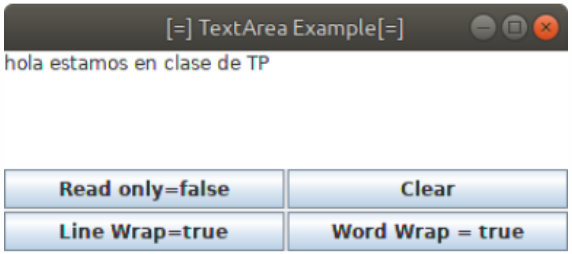
  textarea.setWrapStyleWord(true);

- El tratamiento de eventos se realiza con:

  public void actionPerformed(ActionEvent e)

- Segun se va escribiendo el área de texto, puede ocurrir que parte del texto se deje de visualizar. Por ello es habitual poner un **scroll pane**.

JTextArea

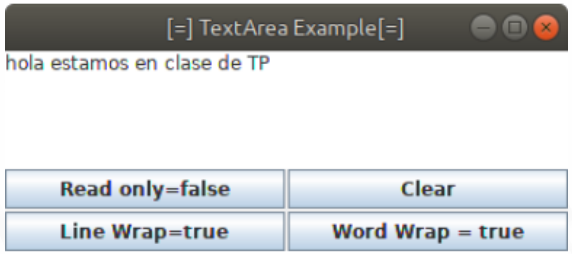



```
private void initGUI() {
  ...
  textArea = new JTextArea(5, 30);
  textArea.setEditable (true);
  textArea.setLineWrap(true);
  textArea.setWrapStyleWord(true);
  ...
}
```

- Se crea un área de texto de 5 filas y 30 columnas
- Es editable, se puede escribir
- La líneas se cortan en función del ancho del área de texto
- Las palabras no se cortan. La palabra pasa completa a otra línea

JTextArea

```
public void actionPerformed(ActionEvent e) {
  if (e.getSource() == clearButton) textArea.setText("");
  else if (e.getSource() == readOnlyButton) {
    boolean editable = textArea.isEditable ();
    textArea.setEditable (!editable );
    if (editable ) readOnlyButton.setText("Read only=true");
    else readOnlyButton.setText("Read only=false");
  }
  else if (e.getSource() ==lineWrap){
    boolean lw = textArea.getLineWrap();
    textArea.setLineWrap(!lw);
    if (lw) lineWrap.setText("Line Wrap=true");
    else lineWrap.setText("Line Wrap=false");
  }
  else if (e.getSource() == wordWrap){
    boolean ww = textArea.getWrapStyleWord();
    ww = !ww;
    textArea.setWrapStyleWord(ww);
    if (ww) wordWrap.setText("Word Wrap=true");
    else wordWrap.setText("Word Wrap=false");
  }
}
```



JScrollPane

- Es una barra de desplazamiento que se puede colocar tanto de forma vertical como horizontal.
- Para colocar un **ScrollPane** en una componente basta indicarlo en su constructora:

  new JScrollPane (Componente, posHorizontal, posVertical);

  donde **posHorizontal** y **posVertical** puede ser:
  - JScrollPane.VERTICAL_SCROLLBAR_AS_NEEDED (opción por defecto)
  - JScrollPane.VERTICAL_SCROLLBAR_ALWAYS
  - JScrollPane.VERTICAL_SCROLLBAR_NEVER
  - JScrollPane.HORIZONTAL_SCROLLBAR_AS_NEEDED
  - JScrollPane.HORIZONTAL_SCROLLBAR_ALWAYS
  - JScrollPane.HORIZONTAL_SCROLLBAR_NEVER

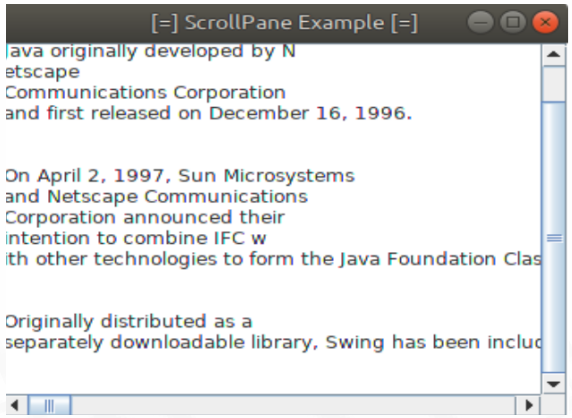
JScrollPane

```
private void initGUI() {
  ...
  JTextArea storyArea = new JTextArea(story);
  storyArea.setEditable (true);

  storyArea.setWrapStyleWord(true);

  JScrollPane area = new JScrollPane(storyArea,
  JScrollPane.VERTICAL_SCROLLBAR_AS_NEEDED,
  JScrollPane.HORIZONTAL_SCROLLBAR_AS_NEEDED);

  area.setPreferredSize (new Dimension(300, 200));
  mainPanel.add(area);
  ...
}
```



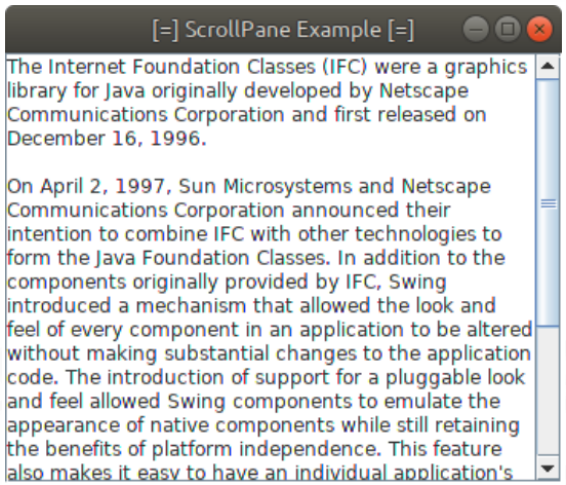
Se añade el Scroll Pane al panel. No hay que añadir el área de texto, ya que está incluido en el scrollPane

JScrollPane

```
private void initGUI() {
  ...
  JTextArea storyArea = new JTextArea(story);
  storyArea.setEditable(true);
  storyArea.setLineWrap(true);
  storyArea.setWrapStyleWord(true);

  JScrollPane area = new JScrollPane(storyArea,
  JScrollPane.VERTICAL_SCROLLBAR_AS_NEEDED,
  JScrollPane.HORIZONTAL_SCROLLBAR_AS_NEEDED);

  area.setPreferredSize(new Dimension(300, 200));
  mainPanel.add(area);
  ...
}
```



Si se añade setLineWrap nunca se creará el ScrollPane horizontal

JScrollPane con JPanel

```
private void initGUI() {
  ...
  JPanel mainPanel = new JPanel(new BorderLayout());

  JPanel p = new JPanel();
  p.setLayout(new BoxLayout(p, BoxLayout.Y_AXIS));

  addButton("Holaa", p);
  addButton("Holaaa", p);
  addButton("Holaaaa", p);
  addButton("Holaaaaa", p);
  addButton("Holaaaaaaaaaaaaaaaaaaaa", p);

  JScrollPane area = new JScrollPane(p,
      JScrollPane.VERTICAL_SCROLLBAR_AS_NEEDED,
      JScrollPane.HORIZONTAL_SCROLLBAR_AS_NEEDED);

  mainPanel.add(area);
  ...
}
```



Se añade el Scroll Pane al panel. No hay que añadir el JPanel, ya que está incluido en el scrollPane

JSeparator

- Dibuja una línea que sirve para dividir componentes.
- Un separador se crea con:

  new JSeparator(SwingConstants.CONSTANTE)

  donde **CONSTANTE** puede ser: **HORIZONTAL**, **VERTICAL**, **CENTER**, **RIGHT**, etc.
- El color de un separador puede fijarse con:

  separador.setBackGround(Color.COLOR);

JSeparator

```
private void initGUI() {
  ...
  for (int i = 0; i < 7; i++) {
    cb[i] = new JCheckBox(places[i]);
    cb[i].addItemListener(this);
    cbPanel.add(cb[i]);


    cbPanel.add(sep);
  }

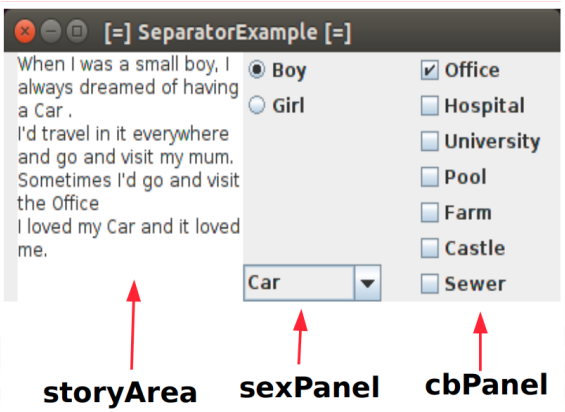
  // No separamos los paneles
  mainPanel.add(storyArea);



  mainPanel.add(sexPanel);



  mainPanel.add(cbPanel);
  ...
}
```

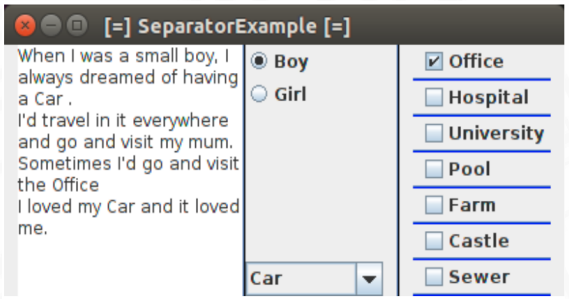


Sin Separadores!

JSeparator

```
private void initGUI() {
  ...
  for (int i = 0; i < 7; i++) {
    cb[i] = new JCheckBox(places[i]);
    cb[i].addItemListener(this);
    cbPanel.add(cb[i]);
    JSeparator sep = new JSeparator(SwingConstants.HORIZONTAL);
    sep.setBackground(Color.blue);
    cbPanel.add(sep);
  }

  // separadores mainPanel
  mainPanel.add(storyArea);
  JSeparator sep = new JSeparator(SwingConstants.VERTICAL);
  sep.setBackground(Color.black);
  mainPanel.add(sep);
  mainPanel.add(sexPanel);
  JSeparator sep1 = new JSeparator(SwingConstants.VERTICAL);
  sep1.setBackground(Color.black);
  mainPanel.add(sep1);
  mainPanel.add(cbPanel);
  ...
}
```



Con Separadores!